

FLARE-On 12 Challenge 2: Project Chimera

By Ana Martinez Gomez

Step 1: Recreating Dr. Khem's Lab Environment

The very first step is to execute the provided `project_chimera.py`. If run with any Python version older than 3.12, the script immediately fails with a `SyntaxError`:

```
None
File "C:\Users\flare\Desktop\project_chimera.py", line 30
    print(f"Bootting up {f"Project Chimera"} from Dr. Khem's journal...")
                        ^^^^^^^^
SyntaxError: f-string: expecting '}'
```

The `SyntaxError` points to a nested f-string that uses double quotes (") inside an f-string delimited by double quotes. This syntax is only valid in Python 3.12+ hinting that the challenge must be run with a 3.12 interpreter.

Commenting out this line and running the script with a version different than 3.12 is also likely to fail due when trying to unmarshal the bytecode, as the marshal format is version-dependent:

```
None
Traceback (most recent call last):
  File "C:\Users\flare\Desktop\project_chimera.py", line 33, in <module>
    exec(marshal.loads(sequencer_code))
ValueError: bad marshal data (unknown type code)
```

Finally, running the script successfully also requires installing several non-standard libraries: [emoji](#), [cowsay](#), [pyjokes](#), and [rc4](#). This is an intentional part of the reversing process.

```
None
python3.12 -m pip install emoji cowsay pyjokes arc4 art
```

```

Booting up Project Chimera from Dr. Khem's journal...
--- Calibrating Genetic Sequencer ---
Decoding catalyst DNA strand...
Synthesizing Catalyst Serum...
--- Catalyst Serum Injected ---
Verifying Lead Researcher's credentials via biometric scan...

A U T H E N T I C A T I O N F A I L E D

Impostor detected, my genius cannot be replicated!
The resulting specimen has developed an unexpected, and frankly useless, sense of humor.

How do you generate a random string? Put a first
year Computer Science student in Vim and ask them
to save and exit.

=====

```

The `project_chimera.py` script contains a zlib-compressed and marshalled Python code object in the `encrypted_sequencer_data` variable that executes via `marshal.loads(sequencer_code)`. To analyze this first payload, we intercept the code object before it's executed and write it to a `.pyc` file for easy decompilation.

Page 2 of 16

```
import zlib

encrypted_sequencer_data =
b'x\x9cm\x96K\xcf\xe2\xe6\x15\xcf\xfd\xcedf\x92\xe6\xd2J\x93\xce...'

decompressed = zlib.decompress(encrypted_sequencer_data)
co = marshal.loads(decompressed)

# Convert Marshall code object `co` to PYC file
# https://stackoverflow.com/a/73454818/6245337
pyc_data = importlib._bootstrap_external._code_to_timestamp_pyc(co)

# Write PYC file to disk
with open("payload1.pyc", "wb") as f:
    f.write(pyc_data)
```

Now we can use a Python decompiler like [pycdc](#) to decompile the written PYC file, recovering the source code of the first payload

Python

```
# Source Generated with Decompyle++
# File: payload1.pyc (Python 3.12)
```

```
import base64
import zlib
import marshal
import types
encoded_catalyst_strand =
b'c$|e+0>7&-6`m!Rzak~l1E|2<;!(^*VQn#qEH||x2b$*W=zw8NW~2mgIMj3sFjzy%<NJQ84^$vqeTG&mC+yhlE677j
-8)F4nD>~?<GqL64o1vBs$bZ4{qE;{|=p@M4Abeb^*>CzIprJ_rCXLX1@k)54$HHULnIe5P-1)Ahj!*6w{D~1%XMwDPu#
jDYhX^DN{q5Q|5-Wq%1@lBx}}|vN1p~UI8h)0U&nS13Dg}x8K^E-(q$P0}4!ly-%m{0Hd>^+3*<0{*s0K-1k|}BLHWKJw
eQrNz5{%F-;@E_{d+ImT17-o7&}0{%uba}w1RL*UARX*79t+0<^B?zm10DX9|2bzp_ztwjy_TdKb)1%eP4d-Xti0Ygjk_
%w!^%1xuMNV4Z8&(*Ue7_^Fby1n3;+G<VDAfqI^h1>0@=Eki5!M~rms%afx`+uxa0*;FzudpqNln5M<@!0qndZ)R<vh4u
&gpmmaMewbT0RJby?(fa7XW#r>ZQ4UE&u|~lZsEY~-lpfWMf0_+pV-H`PXInpwmyo~mZ`tfUK?($KHa%mvNlovZ;Y)D+
e6uw+mY6LNB2Y9&akbWpZ@lh=Si<!J@t|CG86E`)jp!14xEY(h7@$11A4}B9dpL*j)eL{vVcbyMx5_{b13)N@wa~epS8Z
fo&V_Y#fM*g9;@6%j=%iWB0=QS3ewj@0~B!iibu<MqrrJIH{m&FoAGB3#0Nf;x!~dvQ|9#3c})IL6kEvhByJvA{B9%Uq
X0Tg*-+Ak~NW&RJbB?a6weENW&rZrI2ZB!647HW1A^rG4gvj3Yteo30&*};59;7nJF7eh7vjEXwwwPWWzD*3<IvZS#1IL
(1*?u$;EGifKfLDpVb*rXLyw!AP~ZT^~S=4X{31tqe<01kwG$gBZnu8eva3~6;4CxrCH1{Qg{M;GT5@Bdqt%s{XkT;Dya
Bk)v>cTr#=XM@cQ-VZZJ1azh{1Df~fwf(mdYk_cEC``#zrevUuf1-I7DHKqx9c7Me?*iNur9a3~o)A1AmHbK!6#k<d+Qm
XjoUlRAc=R-8EfEvn$TP%?Zb2%`-;wF2Z7c~Qh!QUp%F7d(Q;It@n131iwc^NCTTrj*OW)bEH>BY1Q$YmihSV2QDxrCs
KNT0EmsNif~;-ILG+1$@~sMDCnEHYIbjb?L-swo>NNY60QJ5`2LX(&$Cff*W(c17t80939@QH+>;!kK4jMTi0QA}zM@d
S+wmk4?RtsqIs(NtuZr(Ewj<zXxVots!6<)UP5>nNp1gfkes4T*zd{)6h-GF4>NSQ0}R*91{c`k!=D-D}baN$1fuVNrU
DvGiYVXWYBI456{mCG`ukuZfpN)A<xyb=s}byE(DvZfmpRkvo4CMg+F*3Cf6#?m{g@T4u-G<~mB~wGxg;NVMFDj&f5<)
qG1#7x1YdFEQ_jHRu*e&FUmQ1J<Gp}4$Xq@ya1C(x)S-FIEgQe+IxARLJPRm@DXx&t+<h5L00RJ<E<cw}6ln6?exLHy}9
_dE4pz17oL(~E`{a`E-no7?`5)pDEpNY(-6VaJ?C^<J9(GN!A;n`PTPDZBE;WN>5k=ams`uyy<xmZYd@0g|04|1U(*1PG
```

```

LR>h3WX?aZWQf~69?j-FsML^GvInrgidoM2}r1u&}XB+q}oGg-NR#n^X*4uqBy?1qY$4<jzMBhXA);zPfx3*xU!VW$#ff
a&MCOFRHVn0%6k8aaRw9dY?)7!uP!nGHEb#k+JxY|2h>kX{N{%!`IfvPX|S@e!nA3Iy~#cKVr}%cFx{mYSGj9h1H_Q6ed
khuGk)3Z9gWp`~mJzG74m7(!J^o(!2de`m0?3IDzcV;$RQ`@foiYHlj%{3;+>#iT|K>v-`YH)PTx#fRu(|@AsKT#P^)cn
a!|9sUyU-MtAxP}M>w|Cc1s4_KI9hlp2y|UAEJ$C2$40h6~@uj-!Y-5tEyI$Y%KECN4u6l<*>fcwR_fD^|>djDIJ5u!>A
&1N9itm{<3o-un;-)89^#pIPd{VwyzH_1W0yqZ$H)k$XXD-xcUafgjb=N#i!+0nn-Tj-cEob+(!(B0Wa>FtC;21DH{%^I
Ho=c%;r;jstN15qS_U^F=Ab$c50h5W?fY!%^vdXfE>5Yf!rHF^<aF`B*be*L=(CF(%-E<?)%b0$BJ)|f2ZjG%ISw+Z8Xc
C`j+)bpk<79YXWEkdaV7mwG_ki0baNYym&C&ix(EpA7N#?)|aRxAsRm;!2e%e)a4AvZnHUPvwCa?b&0iHoo'
print('--- Calibrating Genetic Sequencer ---')
print('Decoding catalyst DNA strand...')
compressed_catalyst = base64.b85decode(encoded_catalyst_strand)
marshalled_genetic_code = zlib.decompress(compressed_catalyst)
catalyst_code_object = marshal.loads(marshalled_genetic_code)
print('Synthesizing Catalyst Serum...')
catalyst_injection_function = types.FunctionType(catalyst_code_object, globals())
catalyst_injection_function()

```

Step 3: Breaching the Catalyst Core (Payload 2)

The first payload contains a Base85 encoded, zlib-compressed and marshalled Python code object in the `encoded_catalyst_strand` variable that executes running the `catalyst_injection_function` function. Similarly as we did to extract the first payload, we intercept the code object before it's executed and write it to a `.pyc` file:

```

Python
import base64
import importlib
import marshal
import types
import zlib

encoded_catalyst_strand =
b" c$|e+U2Gf25#Bo<e?>~9Wsx ye#jA#`%-UiSS#D}YR!mcr0q*r`k%}8C0dc-tQ77LIvv-t3h6L2c=noK}Hu@vCKmoT;G
3=&&C<;F|Xn{VKEU8L3t3aFRp?)*sKJ*7@yGw~Q6DsWi+*!`f&3yCC?Cy08x*1#{KE6(d4I1!MkwJD(GFhf2*(}?VmaL^
Exh&U`~vN9ZCP70+0u{dS=pgDvrYzxmga^wvVgce-{h;R|de`0k`CVpDGN8nSxrc0`3Nt1bU5HHZ-7ne6?;VO(wE%FD*b
~G|mBg$V2GceYE1>P)X}K<~rLA3U=i1|TQLW1Pd%#bE&W>m6bzt<h|5ey_oLTVc{Hhki({~>Q>s_?9E#Dy(c$=X701Q)0
hSf?ii}os4;Y9n@V*uP>@E0qV3ZQdbFnk@KhYs-R5S{m2ciB$VpZ;StkSBMScBnV2s<mQW4fMvTF1~#~BfYB~-fYHj2kq
w$K<1a4%HR_n0sc>Z{hyiNz=8RByR=6;p5XzWed3{egKf`MyAkYIfBpdL!y*WRs^dwdU1rI{ _QqIe^;qv}#nDFjzcF??Y
#(J;ZpS<Coj3?>H^a_ueulbNULd{p?%AJNt}$VMN<U1r%JA95NK`{-?b*4db4gFi<xcCA57ZxUKJ}xY9%GN>fB3Br|2_G
u+<Gkd`#=8X!iOK9zVN&Myg#dykB*0z!x(Sya!EH}8sVY>6Y44w3?xFbRzCA8!qRd{Qu8p0co8l9r0LLL3m7u0!AWI4T
+a^xF+%NiliyXz>)xUfmb7uh`@AZNH_LL@_Rf1+DANyFfa4>x8N*Q)Me`g&c3R*~)IR(U=W)0q^=aas;~)~WgmWtET!&%
u*GB}&zX#5GJ;MI4hZ2t2d0JnG; )H5(TwSw&Lb7QR1qal5HA?g8gH0n+a@wCYXX5i6X{sdJ)+an{uTUV9n^x!`sBqsv+q
UUd-K-wx0&1Leq#IaG*SISr+IvRD8S_3KK%atU&~CGVKMViid!_Q_-oD`FJXmh($Arms12+dNCG@r_-iYS9n>bN9eMq8
s+J8pP&j}ie_TxBQVjT(<nhfQ<o*IkrTh<r-_f0N9H7zm!&eT$}3o-<$|#1NkNz=q?eybKwg2wU=5AJ%nE`!B9$`WvWAT

```

```
S%tJ|^a+L!;Q0JnFWPG+fKx&YnZfF>a$Y?df38<7Hp-C!|lJlyn8Sn~%7!_oR@QK9H@>gjTjP#;xP)vF02?UJ_R|t9$ts
z+}&{R}dq+IVi<y8?bV_v~mtavxGSc-E1gfeds4c^lqN3r5$S*OXQ07?EA@gd!$mfyipkebGE<L3ERSu^~MUes##u}c2
i|SQXyQV@K2~!xP<WM)rE0J)(v=*?W8YZjhrX!CGS<6eRX+x@@i095GDH%vNok%TGG7V(fdA+0x112(BBUB(*uwmMamJ>
{S&$Nq#-BN*Q$%Hzw3EDNCFPRo1No$1=wX0|^SdLCjm$1U)oXRUi>!Vt&XcUS@PABc4=1m)+MAQ`GW<`?(WLoR+ZgS`?$
p|~mBguG1Hn$K@$H`56KE9AkC+1Vp%;G}aJQ7dE7G8TL6Q9k+&L=L#7oxejm1rNACZncXC}K=_$Pt>c*-yuuF6C9;C}M(
_a#Pez4{5K-97#edkS;TQ`*|TTx8FD>*TQe-Rf#mSt~>X(-rj@heQWx9b{d`}`T@GPmYDs$wGHDnZsZXQMivTV>~hYU>v
)qpu|p?`CU;?XaA@Pjn=gL%+}yoeQ>NX7+z2Q%?A9#^<eUsiTisM@?s;HYz!sWQ@bwTcgDXxev50kMz**S;_1BL|N7jv
%e&+8Zx4*rMyLKTFty_y+;H7=)rL>qPwP(?s$14{!G)m*qWSbdcV92L$_aVz=<24?a=dEq0{xy>DpAf5z1_Zmg=FUM(B
;2ca%Wz1@`<zYqN{>;l)kwtGjL211NcP^u}5qFEqUuTa!Z5`{r)o>8-$6Juvo*!@ssR19LaHogmx_9<K+F1f&g)E%vWJ
2C#=@|HhR_ebun{k(dodjE3c+4yg-Z0vb<&tGoLzt-?)o8BdwhU2E=mhbz0+n)X%x36~S%*$h(Me!^`d|7WJ(r@-&7ER
Cm|*56xDQUo0r?Y1WBRd&!ZqDq-d)WrZ*j?mmwad!!vXkq3ay&fmUC)>SkJSQN{GD>oGmS_iF5zRy46FqZxd<btjS1PL
bh}g<+VFfb#)xJpk?p!1Dm~w@25Xz|hC8FEB4XN-=<OuiL-b_qD#dd!*q$X(0iu3VHp%muIXy"
```

```
decompressed = zlib.decompress(base64.b85decode(encoded_catalyst_strand))
co = marshal.loads(decompressed)
```

```
# Convert Marshall code object `co` to PYC file
# https://stackoverflow.com/a/73454818/6245337
pyc_data = importlib._bootstrap_external._code_to_timestamp_pyc(co)

# Write PYC file to disk
with open("payload2.pyc", "wb") as f:
    f.write(pyc_data)
```

But in this case using a decompiler like [pycdc](#) to decompile the `.pyc` file fails due to the usage of unsupported Python constructs and opcodes.

```
Python
# Source Generated with Decompyle++
# File: payload2.pyc (Python 3.12)

import os
import sys
import emoji
import random
import asyncio
import cowsay
import pyjokes
import art
from arc4 import ARC4

async def activate_catalyst():
    Unsupported opcode: RETURN_GENERATOR (109)
    pass
# WARNING: Decompyle incomplete
```

```
asyncio.run(activate_catalyst())
```

The failure of decompilers forces us to use a disassembler like [pycdas](#). The complete pycdas output is provided in this document in Appendix 1.

By analyzing the disassembled Python bytecode, we can reconstruct the program's logic. But analyzing the raw bytecode from a disassembler is significantly more challenging than reading source code. Luckily for us, AI tools like Gemini can assist in converting the complex bytecode back into readable Python source code: <https://g.co/gemini/share/126f4a1d8c31> Gemini Pro tends to work better for tasks like writing code and disassembling Python bytecode, but for this case Gemini Flash works well too if you ask the right question. Give it a second (or third) try if it does not work the first time!

```
Python
import os
import sys
import emoji
import random
import asyncio
import cowsay
import pyjokes
import art
from arc4 import ARC4

async def activate_catalyst():
    LEAD_RESEARCHER_SIGNATURE = b'm\x1b@I\x1dAoe@\x07ZF[BL\rN\n\x0cS'
    ENCRYPTED_CHIMERA_FORMULA =
b'r2b-\r\x9e\xf2\x1fp\x185\x82\xcf\xfc\x90\x14\xf10\xad#]\xf3\xe2\xc0L\xd0\xc1e\x0c\xea\xec\x
ae\x11b\xa7\x8c\xaa!\xa1\x9d\xc2\x90'
    print('--- Catalyst Serum Injected ---')
    print("Verifying Lead Researcher's credentials via biometric scan...")
    current_user = os.getlogin().encode()
    user_signature = bytes((c ^ i + 42) for i, c in enumerate(current_user))
    await asyncio.sleep(0.01)
    status = 'pending'
    if status == 'pending':
        if user_signature == LEAD_RESEARCHER_SIGNATURE:
            art.tprint('AUTHENTICATION SUCCESS', font='small')
            print('Biometric scan MATCH. Identity confirmed as Lead Researcher.')
            print('Finalizing Project Chimera...')
            arc4_decipher = ARC4(current_user)
            decrypted_formula = arc4_decipher.decrypt(ENCRYPTED_CHIMERA_FORMULA).decode()
            cowsay.cow('I am alive! The secret formula is:\n' + decrypted_formula)
```

```

        return None
    else:
        art.tprint('AUTHENTICATION   FAILED', font='small')
        print('Impostor detected, my genius cannot be replicated!')
        print('The resulting specimen has developed an unexpected, and frankly useless,
sense of humor.')
        joke = pyjokes.get_joke(language='en', category='all')
        animals = cowsay.char_names[1:]
        print(cowsay.get_output_string(random.choice(animals), pyjokes.get_joke()))
        sys.exit(1)
    else:
        print('System error: Unknown experimental state.')
        return None

asyncio.run(activate_catalyst())

```

Step 4: Reversing the Biometric Signature

The source code for Payload 2 contains the core logic of the challenge. The script gets the current user's login name, encodes it using XORs with a key based on the character index, and compares it to the hardcoded value in the `LEAD_RESEARCHER_SIGNATURE` variable: `b'm\x1b@I\x1dAoe@\x07ZF[BL\rN\n\x0cS'`.

The encoding algorithm is:

```

Python
user_signature = bytes(c ^ (i + 42) for i, c in enumerate(current_user))

```

Because the XOR operation is its own inverse, we can recover the original username by applying the exact same algorithm to the `LEAD_RESEARCHER_SIGNATURE` value:

```

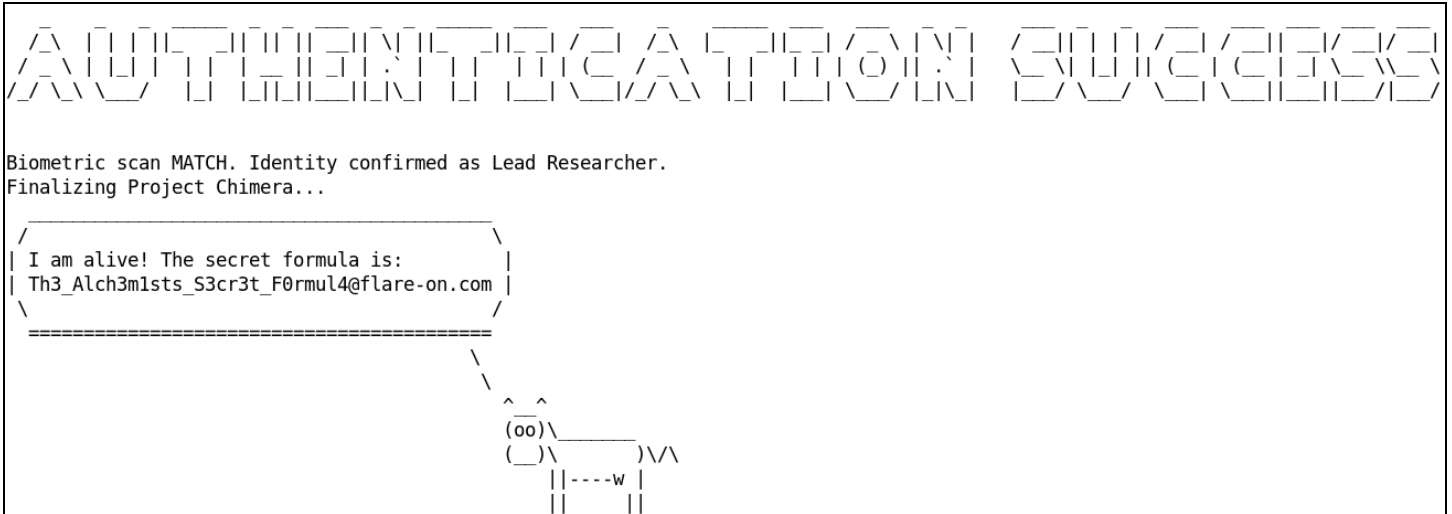
Python
LEAD_RESEARCHER_SIGNATURE = b'm\x1b@I\x1dAoe@\x07ZF[BL\rN\n\x0cS'
original_username = bytes(c ^ (i + 42) for i, c in enumerate(LEAD_RESEARCHER_SIGNATURE))
print(f"The required username is: {original_username.decode()}")

```

Running this script yields the required username: `G0ld3n_Tr4nsmut4t10n`.

Step 5: Finalizing the Experiment and Claiming the Flag

Set the system's username to `G0ld3n_Tr4nsmut4t10n` and run `project_chimera.py` to get the flag:
`Th3_Alch3m1sts_S3cr3t_F0rmul4@flare-on.com`



Alternatively to setting the system's username, create a simple script that hardcodes the correct username and runs only the successful decryption path from Payload 2's logic:

```

Python
import cowsay
import art
from arc4 import ARC4

current_user = b'G0ld3n_Tr4nsmut4t10n'
ENCRYPTED_CHIMERA_FORMULA =
b'r2b-\r\x9e\xfa\x1fp\x185\x82\xcf\xfc\x90\x14\xfa\xad#]\xf3\xe2\xc0L\xd0\xc1e\x0c\xea\xec\x
ae\x11b\xa7\x8c\xaa!\xa1\x9d\xc2\x90'

art.tprint('AUTHENTICATION SUCCESS', font='small')
print('Biometric scan MATCH. Identity confirmed as Lead Researcher.')
print('Finalizing Project Chimera...')

arc4_decipher = ARC4(current_user)
decrypted_formula = arc4_decipher.decrypt(ENCRYPTED_CHIMERA_FORMULA).decode()

cowsay.cow('I am alive! The secret formula is:\n' + decrypted_formula)
  
```

Appendix 1: pycdas Program Output

```
None
payload2.pyc (Python 3.12)
[Code]
  File Name: <catalyst_core>
  Object Name: <module>
  Qualified Name: <module>
  Arg Count: 0
  Pos Only Arg Count: 0
  KW Only Arg Count: 0
  Stack Size: 4
  Flags: 0x00000000
  [Names]
    'os'
    'sys'
    'emoji'
    'random'
    'asyncio'
    'cowsay'
    'pyjokes'
    'art'
    'arc4'
    'ARC4'
    'activate_catalyst'
    'run'
  [Locals+Names]
  [Constants]
    0
    None
    (
      'ARC4'
    )
  [Code]
    File Name: <catalyst_core>
    Object Name: activate_catalyst
    Qualified Name: activate_catalyst
    Arg Count: 0
    Pos Only Arg Count: 0
    KW Only Arg Count: 0
    Stack Size: 7
    Flags: 0x00000083 (CO_OPTIMIZED | CO_NEWLOCALS | CO_COROUTINE)
    [Names]
      'print'
      'os'
      'getlogin'
```

```

'encode'
'bytes'
'enumerate'
'asyncio'
'sleep'
'art'
'tprint'
'ARC4'
'decrypt'
'decode'
'cowsay'
'cow'
'pyjokes'
'get_joke'
'char_names'
'get_output_string'
'random'
'choice'
'sys'
'exit'
[Locals+Names]
'LEAD_RESEARCHER_SIGNATURE'
'ENCRYPTED_CHIMERA_FORMULA'
'current_user'
'user_signature'
'status'
'arc4_decipher'
'decrypted_formula'
'joke'
'animals'
[Constants]
None
b'm\x1b@I\x1dAoe@\x07ZF[BL\rN\n\x0cS'

b'r2b-\r\x9e\xf2\x1fp\x185\x82\xcf\xfc\x90\x14\xf10\xad#]\xf3\xe2\xc0L\xd0\xc1e\x0c\xea\xec\x
ae\x11b\xa7\x8c\xaa!\xa1\x9d\xc2\x90'
'--- Catalyst Serum Injected ---'
"Verifying Lead Researcher's credentials via biometric scan..."
[Code]
File Name: <catalyst_core>
Object Name: <genexpr>
Qualified Name: activate_catalyst.<locals>.<genexpr>
Arg Count: 1
Pos Only Arg Count: 0
KW Only Arg Count: 0
Stack Size: 4
Flags: 0x00000033 (CO_OPTIMIZED | CO_NEWLOCALS | CO_NESTED |
CO_GENERATOR)

```

```

[Names]
[Locals+Names]
    '.0'
    'i'
    'c'
[Constants]
    42
    None
[Disassembly]
    0      RETURN_GENERATOR
    2      POP_TOP
    4      RESUME
    6      LOAD_FAST
    8      FOR_ITER
    12     UNPACK_SEQUENCE
    16     STORE_FAST
    18     STORE_FAST
    20     LOAD_FAST
    22     LOAD_FAST
    24     LOAD_CONST
    26     BINARY_OP
    30     BINARY_OP
    34     YIELD_VALUE
    36     RESUME
    38     POP_TOP
    40     JUMP_BACKWARD
    42     END_FOR
    44     RETURN_CONST
    46     CALL_INTRINSIC_1
(INTRINSIC_STOPITERATION_ERROR)
    48     RERAISE
0.01
'pending'
'AUTHENTICATION    SUCCESS'
'small'
(
    'font'
)
'Biometric scan MATCH. Identity confirmed as Lead Researcher.'
'Finalizing Project Chimera...'
'I am alive! The secret formula is:\n'
'AUTHENTICATION    FAILED'
'Impostor detected, my genius cannot be replicated!'
'The resulting specimen has developed an unexpected, and frankly useless,
sense of humor.'
'en'
'all'
(

```

```

        'language'
        'category'
    )
    1
    'System error: Unknown experimental state.'
[Disassembly]
    0      RETURN_GENERATOR
    2      POP_TOP
    4      RESUME                0
    6      LOAD_CONST            1:
b'm\x1b@I\x1dAoe@\x07ZF[BL\rN\n\x0cS'
    8      STORE_FAST            0: LEAD_RESEARCHER_SIGNATURE
   10     LOAD_CONST            2:
b'r2b-\r\x9e\xf2\x1fp\x185\x82\xcf\xfc\x90\x14\xf10\xad#]\xf3\xe2\xc0L\xd0\xc1e\x0c\xea\xec\x
ae\x11b\xa7\x8c\xaa!\xa1\x9d\xc2\x90'
   12     STORE_FAST            1: ENCRYPTED_CHIMERA_FORMULA
   14     LOAD_GLOBAL            1: NULL + print
   24     LOAD_CONST            3: '--- Catalyst Serum Injected ---'
   26     CALL                   1
   34     POP_TOP
   36     LOAD_GLOBAL            1: NULL + print
   46     LOAD_CONST            4: "Verifying Lead Researcher's
credentials via biometric scan..."
   48     CALL                   1
   56     POP_TOP
   58     LOAD_GLOBAL            3: NULL + os
   68     LOAD_ATTR              4: getlogin
   88     CALL                   0
   96     LOAD_ATTR              7: encode
  116     CALL                   0
  124     STORE_FAST            2: current_user
  126     LOAD_GLOBAL            9: NULL + bytes
  136     LOAD_CONST            5: <CODE> <genexpr>
  138     MAKE_FUNCTION          0
  140     LOAD_GLOBAL            11: NULL + enumerate
  150     LOAD_FAST              2: current_user
  152     CALL                   1
  160     GET_ITER
  162     CALL                   0
  170     CALL                   1
  178     STORE_FAST            3: user_signature
  180     LOAD_GLOBAL            13: NULL + asyncio
  190     LOAD_ATTR              14: sleep
  210     LOAD_CONST            6: 0.01
  212     CALL                   1
  220     GET_AWAITABLE          0
  222     LOAD_CONST            0: None
  224     SEND                   3 (to 232)

```

```

228     YIELD_VALUE                2
230     RESUME                     3
232     JUMP_BACKWARD_NO_INTERRUPT 5 (to 224)
234     END_SEND
236     POP_TOP
238     LOAD_CONST                 7: 'pending'
240     STORE_FAST                4: status
242     LOAD_FAST                 4: status
244     LOAD_CONST                 7: 'pending'
246     COMPARE_OP                40 (==)
250     POP_JUMP_IF_FALSE        294 (to 842)
254     LOAD_FAST                 3: user_signature
256     LOAD_FAST                 0: LEAD_RESEARCHER_SIGNATURE
258     COMPARE_OP                40 (==)
262     POP_JUMP_IF_FALSE        112 (to 488)
264     LOAD_GLOBAL               17: NULL + art
274     LOAD_ATTR                18: tprint
294     LOAD_CONST                8: 'AUTHENTICATION    SUCCESS'
296     LOAD_CONST                9: 'small'
298     KW_NAMES                  10: ('font',)
300     CALL                      2
308     POP_TOP
310     LOAD_GLOBAL               1: NULL + print
320     LOAD_CONST                11: 'Biometric scan MATCH. Identity
confirmed as Lead Researcher.'
322     CALL                      1
330     POP_TOP
332     LOAD_GLOBAL               1: NULL + print
342     LOAD_CONST                12: 'Finalizing Project Chimera...'
344     CALL                      1
352     POP_TOP
354     LOAD_GLOBAL               21: NULL + ARC4
364     LOAD_FAST                2: current_user
366     CALL                      1
374     STORE_FAST               5: arc4_decipher
376     LOAD_FAST                5: arc4_decipher
378     LOAD_ATTR                23: decrypt
398     LOAD_FAST                1: ENCRYPTED_CHIMERA_FORMULA
400     CALL                      1
408     LOAD_ATTR                25: decode
428     CALL                      0
436     STORE_FAST               6: decrypted_formula
438     LOAD_GLOBAL               27: NULL + cowsay
448     LOAD_ATTR                28: cow
468     LOAD_CONST                13: 'I am alive! The secret formula
is:\n'
470     LOAD_FAST                6: decrypted_formula
472     BINARY_OP                 0 (+)

```

```

476     CALL                                1
484     POP_TOP
486     RETURN_CONST                       0: None
488     LOAD_GLOBAL                         17: NULL + art
498     LOAD_ATTR                           18: tprint
518     LOAD_CONST                         14: 'AUTHENTICATION   FAILED'
520     LOAD_CONST                           9: 'small'
522     KW_NAMES                           10: ('font',)
524     CALL                                2
532     POP_TOP
534     LOAD_GLOBAL                         1: NULL + print
544     LOAD_CONST                         15: 'Impostor detected, my genius
cannot be replicated!'
546     CALL                                1
554     POP_TOP
556     LOAD_GLOBAL                         1: NULL + print
566     LOAD_CONST                         16: 'The resulting specimen has
developed an unexpected, and frankly useless, sense of humor.'
568     CALL                                1
576     POP_TOP
578     LOAD_GLOBAL                         31: NULL + pyjokes
588     LOAD_ATTR                           32: get_joke
608     LOAD_CONST                         17: 'en'
610     LOAD_CONST                         18: 'all'
612     KW_NAMES                           19: ('language', 'category')
614     CALL                                2
622     STORE_FAST                           7: joke
624     LOAD_GLOBAL                         26: cowsay
634     LOAD_ATTR                           34: char_names
654     LOAD_CONST                         20: 1
656     LOAD_CONST                           0: None
658     BINARY_SLICE
660     STORE_FAST                           8: animals
662     LOAD_GLOBAL                         1: NULL + print
672     LOAD_GLOBAL                         27: NULL + cowsay
682     LOAD_ATTR                           36: get_output_string
702     LOAD_GLOBAL                         39: NULL + random
712     LOAD_ATTR                           40: choice
732     LOAD_FAST                           8: animals
734     CALL                                1
742     LOAD_GLOBAL                         31: NULL + pyjokes
752     LOAD_ATTR                           32: get_joke
772     CALL                                0
780     CALL                                2
788     CALL                                1
796     POP_TOP
798     LOAD_GLOBAL                         43: NULL + sys
808     LOAD_ATTR                           44: exit

```

```

      828      LOAD_CONST          20: 1
      830      CALL                1
      838      POP_TOP
      840      RETURN_CONST        0: None
      842      NOP
      844      LOAD_GLOBAL          1: NULL + print
      854      LOAD_CONST          21: 'System error: Unknown
experimental state.'
      856      CALL                1
      864      POP_TOP
      866      RETURN_CONST        0: None
      868      CLEANUP_THROW
      870      JUMP_BACKWARD        320 (to 234)
      874      CALL_INTRINSIC_1     3 (INTRINSIC_STOPITERATION_ERROR)
      876      RERAISE              1
[Disassembly]
  0      RESUME                  0
  2      LOAD_CONST              0: 0
  4      LOAD_CONST              1: None
  6      IMPORT_NAME              0: os
  8      STORE_NAME              0: os
 10      LOAD_CONST              0: 0
 12      LOAD_CONST              1: None
 14      IMPORT_NAME              1: sys
 16      STORE_NAME              1: sys
 18      LOAD_CONST              0: 0
 20      LOAD_CONST              1: None
 22      IMPORT_NAME              2: emoji
 24      STORE_NAME              2: emoji
 26      LOAD_CONST              0: 0
 28      LOAD_CONST              1: None
 30      IMPORT_NAME              3: random
 32      STORE_NAME              3: random
 34      LOAD_CONST              0: 0
 36      LOAD_CONST              1: None
 38      IMPORT_NAME              4: asyncio
 40      STORE_NAME              4: asyncio
 42      LOAD_CONST              0: 0
 44      LOAD_CONST              1: None
 46      IMPORT_NAME              5: cowsay
 48      STORE_NAME              5: cowsay
 50      LOAD_CONST              0: 0
 52      LOAD_CONST              1: None
 54      IMPORT_NAME              6: pyjokes
 56      STORE_NAME              6: pyjokes
 58      LOAD_CONST              0: 0
 60      LOAD_CONST              1: None
 62      IMPORT_NAME              7: art

```

64	STORE_NAME	7: art
66	LOAD_CONST	0: 0
68	LOAD_CONST	2: ('ARC4',)
70	IMPORT_NAME	8: arc4
72	IMPORT_FROM	9: ARC4
74	STORE_NAME	9: ARC4
76	POP_TOP	
78	LOAD_CONST	3: <CODE> activate_catalyst
80	MAKE_FUNCTION	0
82	STORE_NAME	10: activate_catalyst
84	PUSH_NULL	
86	LOAD_NAME	4: asyncio
88	LOAD_ATTR	22: run
108	PUSH_NULL	
110	LOAD_NAME	10: activate_catalyst
112	CALL	0
120	CALL	1
128	POP_TOP	
130	RETURN_CONST	1: None